TaskFlow Enterprise API

FastAPI + PostgreSQL | Developer setup, commands, API guide, tests and GitHub handoff

Documentation date: 29 October 2026
Environment: Windows PowerShell, Python 3.13, PostgreSQL, FastAPI
Project folder used during development: E:\Projects\TaskFlow-API

Scope: Authentication, JWT access and refresh tokens, role-based access foundation, teams, team memberships, projects, tasks, filtering, pagination and automated tests. Docker and Redis are intentionally excluded.

Important: This guide records the commands and implementation pattern discussed during development. Some installation steps are reconstructed for a clean clone and may need small adjustments to match your repository files. Confirm your final project dependencies, environment variable names, and endpoint schemas before publishing.

1. Technology stack

Layer	Technology / purpose
Language	Python 3.13 (3.13.14 observed in local test output)
Web framework	FastAPI, Pydantic, Uvicorn
Database	PostgreSQL, psycopg 3 driver
ORM / migrations	SQLAlchemy 2.x, Alembic
Authentication	JWT access tokens, refresh token rotation, password hashing (pwdlib)
Authorization	Authenticated users, team ownership and membership checks
Testing	Pytest, FastAPI TestClient, isolated taskflow_test_db
API documentation	OpenAPI / Swagger UI at /docs, ReDoc at /redoc
Developer workflow	VS Code, PowerShell, Python virtual environment, Git/GitHub

2. Architecture and feature overview

API flow: HTTP request -> FastAPI route -> JWT/auth dependency -> SQLAlchemy session -> PostgreSQL -> Pydantic response.

Data relationships: Users can own teams and belong to multiple teams through team_members. Teams contain projects. Projects contain tasks. Tasks can be assigned to users. Team membership and ownership determine access.

Feature	Key endpoints / behavior
Health	GET /api/health

Feature	Key endpoints / behavior
Authentication	POST /api/v1/auth/register; /login; /refresh; /logout
Current user	GET /api/v1/users/me
Teams	POST, GET /api/v1/teams; GET, PATCH, DELETE /api/v1/teams/{team_id}
Membership	GET, POST /api/v1/teams/{team_id}/members; DELETE /api/v1/teams/{team_id}/members/{user_id}
Projects	POST, GET /api/v1/projects; GET, PATCH, DELETE /api/v1/projects/{project_id}
Tasks	POST, GET /api/v1/tasks; GET, PATCH, DELETE /api/v1/tasks/{task_id}
Filtering	List endpoints support relevant filters, search and page/page_size; confirm exact query fields in Swagger

3. Create a clean local development environment

3.1 Clone from GitHub (after repository is published)

```
git clone https://github.com/YOUR_GITHUB_USERNAME/TaskFlow-API.git
cd TaskFlow-API
```

Replace the URL with your actual GitHub repository URL. The original local project was at E:\Projects\TaskFlow-API.

3.2 Confirm Python and create the virtual environment

```
py -3.13 --version
py -3.13 -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
python --version
```

If the Python launcher cannot locate 3.13, install Python 3.13 and retry. The process-scoped execution policy change affects only that terminal session.

3.3 Install dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The repository should contain a committed requirements.txt generated from the working environment. To create it on the original computer:

```
python -m pip freeze > requirements.txt
```

Avoid generating requirements from a global Python installation. The project environment should be active first.

3.4 Environment configuration

Create a local .env based on .env.example (if provided). Example values, not real secrets:

```
DATABASE_URL=postgresql+psycopg://taskflow_user:<PASSWORD>@localhost:5432/taskflow_db
JWT_SECRET_KEY=<GENERATE_A_LONG_RANDOM_SECRET>
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

```
REFRESH_TOKEN_EXPIRE_DAYS=7
```

These variable names are based on the existing project discussions. Match all required fields in app/core/config.py (or your actual settings module). Never commit .env or .env.test.

3.5 PostgreSQL setup

Install/start PostgreSQL, then connect in pgAdmin as an administrator. Run the following as a template; replace the password locally. If the role/database already exist, do not recreate them.

```
CREATE ROLE taskflow_user WITH LOGIN PASSWORD '<YOUR_LOCAL_PASSWORD>';
CREATE DATABASE taskflow_db OWNER taskflow_user;
CREATE DATABASE taskflow_test_db OWNER taskflow_user;
```

In pgAdmin, connect specifically to each database and ensure the role can create objects in its public schema if needed:

```
GRANT USAGE, CREATE ON SCHEMA public TO taskflow_user;
```

Run the grant inside taskflow_db and taskflow_test_db separately if necessary. Use least privilege outside local development.

3.6 Apply database migrations

```
python -m alembic current
python -m alembic heads
python -m alembic upgrade head
python -m alembic check
```

On the original machine, team models were added with revision 50dba296d375, following 936f11cf5e43. The subsequent project/task migration ID is repository-dependent. After cloning, run upgrade head; do not generate a new initial migration for an existing migration history.

4. Start FastAPI and Swagger UI

```
cd E:\Projects\TaskFlow-API
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

For a cloned project located elsewhere, run from its own root directory instead. Browser links: http://127.0.0.1:8000/docs (Swagger), http://127.0.0.1:8000/redoc (ReDoc). Stop the server with Ctrl+C.

If PowerShell blocks activation or a global Python 3.15 is selected, bypass activation entirely:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

5. Demo users and authentication

Local development demo credentials only. These example passwords are not suitable for production and should not be used for real accounts. Registration may fail with 409/400 if the email already exists; log in instead.

Register the team owner (example)

```
POST /api/v1/auth/register
{
  "name": "Test Employee",
  "email": "employee@example.com",
  "password": "TestPassword123!"
}
```

Register the second user (example)

```
POST /api/v1/auth/register
{
  "name": "Test Developer",
  "email": "developer2@example.com",
  "password": "TestPassword123!"
}
```

User IDs are assigned by the database; in the development screenshots the demo owner was ID 1 and developer ID 2, but IDs will vary after a fresh clone.

Log in and authorize Swagger

```
POST /api/v1/auth/login
{
  "email": "employee@example.com",
  "password": "TestPassword123!"
}
```

Copy the returned access_token. In Swagger, click Authorize and paste the token into the bearer field (typically without typing the word Bearer). The request will send Authorization: Bearer <token>. A missing token returns HTTP 401 Not authenticated.

Refresh and logout

```
POST /api/v1/auth/refresh
{ "refresh_token": "<CURRENT_REFRESH_TOKEN>" }

POST /api/v1/auth/logout
{ "refresh_token": "<CURRENT_REFRESH_TOKEN>" }
```

Refresh-token rotation issues a replacement refresh token; an old token must not be reused. Logout revokes the refresh session. Use the exact schema exposed in Swagger if it differs.

6. Teams, memberships, projects and tasks - sample workflow

6.1 Create a team as an authenticated user

```
POST /api/v1/teams
{
  "name": "Backend Development Team",
  "description": "Handles FastAPI, PostgreSQL and backend services"
}
```

The development run created team ID 1, owned by user ID 1. IDs differ on a fresh database.

6.2 Add a team member (team owner token required)

```
POST /api/v1/teams/1/members
{ "user_id": 2 }

GET /api/v1/teams/1/members
```

Expected behavior: 201 Created for a new membership, 409 for duplicate membership, 401 if not authenticated, and 403 for unauthorized changes by a non-owner.

6.3 Create a project

```
POST /api/v1/projects
{
  "team_id": 1,
```

```
  "name": "TaskFlow Backend Development",
  "description": "Build enterprise project and task APIs",
  "status": "active",
  "start_date": "2026-10-29",
  "due_date": "2026-12-15"
}
```

6.4 Create a task

```
POST /api/v1/tasks
{
  "project_id": 1,
  "title": "Implement task filtering",
  "description": "Add pagination, status and priority filters",
  "status": "todo",
  "priority": "high",
  "assignee_id": 2,
  "due_date": "2026-11-15"
}
```

6.5 Query and update records

```
GET /api/v1/projects
GET /api/v1/projects/1
PATCH /api/v1/projects/1
GET /api/v1/tasks?project_id=1&priority=high&status=todo&page=1&page_size=10
GET /api/v1/tasks/1
PATCH /api/v1/tasks/1
```

Sample filters and payloads reflect the Phase 5 integration instructions. The repository implementation and Swagger OpenAPI schema are the source of truth for supported fields, enums, and response shape.

7. Automated tests

The original test suite used a dedicated PostgreSQL database, taskflow_test_db. Three initial tests were verified passing: health check, registration/login/profile, and unauthorized protected endpoint. Later Phase 5 tests were supplied for integration; their final run results have not been independently inspected in this documentation.

7.1 Configure the test database

```
TEST_DATABASE_URL=postgresql+psycopg://taskflow_user:<PASSWORD>@localhost:5432/taskflow_test_d
    b
```

Put this value in .env.test, never in the tracked repository. Tests must not point to taskflow_db. Check your tests/conftest.py safety guard before running.

7.2 Run tests

```
python -m pytest -v --tb=short
python -m pytest tests/test_auth.py -v
python -m pytest tests/test_phase5.py -v
```

Some test files or endpoint assumptions may differ if local integration was customized. Resolve test failures before claiming a fully passing suite.

8. Development command history

Command	Why we ran it
python --version	Confirm active Python 3.13, not global Python 3.15
python -m pip show uvicorn	Check Uvicorn installation in active environment
python -m pytest	Run authentication test suite
python -m alembic current	Show applied database revision
python -m alembic heads	Show latest migration script
python -m alembic revision --autogenerate -m "create teams and team members"	Generate teams/team_members migration
python -m alembic upgrade head	Apply pending schema changes
python -m alembic check	Check ORM metadata against database schema
python -m uvicorn app.main:app --reload --port 8000	Start local API server

Confirmed team migration history: 936f11cf5e43 -> 50dba296d375. PostgreSQL initially denied CREATE in the test schema; GRANT USAGE, CREATE ON SCHEMA public TO taskflow_user resolved the test setup issue. A separate incident where Alembic hung was resolved after PostgreSQL connectivity was restored.

9. GitHub preparation and upload

9.1 Files to include

Commit app/, alembic/, tests/, alembic.ini, pytest.ini, requirements.txt, README.md, .gitignore, and a sanitized .env.example. Include project/task migration scripts and all imported model files.

9.2 Keep private files out of Git

```
.venv/
.env
.env.*
!.env.example
__pycache__/
.pytest_cache/
*.pyc
.vscode/
*.log
```

Check that the .gitignore patterns work before adding files. Never commit a real DB password, JWT secret, access token, refresh token, or real user data. The demo passwords above are public examples only; do not use them as deployed credentials.

9.3 Initialize and push

```
git init
git add .
git status
git commit -m "Initial TaskFlow Enterprise API release"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/TaskFlow-API.git
git push -u origin main
```

Create the empty GitHub repository before pushing. If the project already has a Git repository, skip git init and adjust the remote. Review git status carefully before committing.

9.4 Recommended README sections

Overview, feature list, technology stack, architecture, prerequisites, installation, environment variables, PostgreSQL setup, Alembic migrations, run commands, Swagger usage, demo accounts, API examples, tests, security notes, and future improvements.

10. Troubleshooting

Symptom	Resolution
No module named uvicorn; Python 3.15 shown	Activate .venv or run .\.venv\Scripts\python.exe directly
Activate.ps1 blocked	Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
Alembic hangs on connect	Check PostgreSQL service, host/port, and DATABASE_URL; test connection with timeout
permission denied for schema public	Grant USAGE, CREATE on public schema of the correct database
401 Not authenticated in Swagger	Log in as the correct user and authorize using the access token
409 User is already a team member	Use a different registered user ID
Database tables missing after clone	Verify DATABASE_URL points to taskflow_db and run alembic upgrade head
Pytest touches production database	Stop; correct .env.test and verify isolation guard

11. Final handoff checklist

☐ Working requirements.txt and sanitized .env.example are committed.

☐ All Alembic migration scripts, including projects/tasks, are committed.

☐ Fresh clone works with Python 3.13 and PostgreSQL.

☐ Swagger /docs loads and register/login/refresh work.

☐ Teams, membership, projects, and tasks work with authorization.

☐ Pytest runs only against taskflow_test_db and passes.

☐ No secrets, virtual environments, or local database files are committed.

☐ GitHub README includes installation steps and screenshots if useful.

Note: This PDF is intended for local setup and portfolio documentation. It does not replace a security review or production deployment configuration.